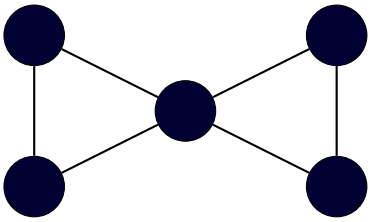
Python 与数据结构

Data Structure



XiaTian

Renmin University of China

2026 年 3 月 18 日

序言

目录

第一章　线性表：从排队到数据组织的艺术 **1**

1.1　什么是线性表？ 1

1.1.1　定义与直观理解 1

1.1.2　核心特征 1

1.2　顺序表：排好座位的电影院 1

1.2.1　原理分析 2

1.2.2　生活中的例子 2

1.2.3　Python 实现 2

1.2.4　顺序表的优缺点 3

1.3　链表：寻宝游戏 3

1.3.1　原理分析 3

1.3.2　节点定义 3

1.3.3　插入操作的艺术 4

1.3.4　为什么需要头节点？ 4

1.3.5　链表的优缺点 4

1.4　更多形式的链表 5

1.4.1　双向链表 5

1.4.2　循环链表 5

1.5　顺序表与链表的选择 5

1.6　实战挑战：约瑟夫环问题 6

1.7　课后作业 7

第二章　栈（Stack）：后进先出的智慧 **9**

2.1　什么是栈？——后进先出的世界 9

2.1.1　核心原则：后进先出（LIFO） 9

2.1.2　为什么是“受限”的？ 9

2.2　栈的基本操作 10

2.2.1　Python 中的栈实现 10

2.3　栈的两种存储结构 10

2.3.1　顺序栈 10

2.3.2　链栈 11

2.4　栈的经典应用 11

2.4.1　撤销操作（Undo） 11

2.4.2 括号匹配 11
2.4.3 表达式求值（逆波兰表达式） 12
2.4.4 函数调用与递归 12
2.5 深度探索：迷宫寻路与回溯 12
2.6 栈与队列的对比 13
2.7 自测练习 13

1 ⟵

线性表：从排队到数据组织的艺术

在计算机的世界里，如果要组织一组同类型的数据，最直观的方式就是把它们排成一队。这种逻辑结构，我们称为线性表。线性表就像生活中的排队：每个人都有自己的位置，除了第一个人和最后一个人，每个人都知道自己前面是谁、后面是谁。

1.1 什么是线性表?

1.1.1 定义与直观理解

线性表是由 n 个具有相同特性的数据元素组成的有限序列。你可以把它想象成：

英文字母表：(A, B, C, ⋯, Z) –每个字母都是一个元素，顺序固定。

奶茶店的排队队列：每个人就是一个数据元素，除了第一个和最后一个，每个人都有一个前排和后排。队伍的长度可以变化，但元素之间的前后关系始终存在。

1.1.2 核心特征

- 唯一性：有且仅有一个第一个元素和一个最后一个元素。
- 有序偶关系：除了头尾，每个元素都有一个前驱（它前面的元素）和一个后继（它后面的元素）。
- 同质性：虽然元素的值可以不同，但同一线性表中的元素必须具有相同的数据类型（比如都是整数，或者都是学生信息）。

现代应用视角：在大语言模型处理文本时，输入的单词序列本质上就是一个线性表。模型会根据当前单词（前驱）来预测下一个可能的单词（后继）。这体现了线性表在人工智能领域的应用。

1.2 顺序表：排好座位的电影院

1.2.1 原理分析

顺序表在计算机内存中申请了一块连续的存储空间，元素一个挨一个地存放。这就像电影院里的一排座位，座位号是连续的（$1, 2, 3, \cdots$）。如果你知道第一个座位的位置，以及每个座位占用的空间大小，那么你就可以直接算出任意座位的位置。

假设第一个元素存储在地址 b，每个元素占用的空间大小为 l，那么第 i 个元素的存储地址就是：

$$\text{Address}(a_i) = b + (i - 1) \times l$$

这个公式意味着，我们可以直接通过下标计算出元素的位置，不需要逐个查找。这就是顺序表“随机访问”快的原因。

1.2.2 生活中的例子

想象你是一个电影院的工作人员，手里有一张座位表。如果有人问你第 5 排的观众是谁，你立刻就能找到。但如果有人要插队到第 2 个座位，那么从第 2 个座位开始，后面所有的人都要往后挪一个位置，非常麻烦。

1.2.3 Python 实现

Python 的内置列表 `list` 就是一种动态的顺序表，它可以根据需要自动扩容。下面我们模拟一个简单的顺序表：

```python
class SequenceList:
    def __init__(self, max_size, elements=None):
        self.max_size = max_size
        self.elements = [None] * max_size  # 预先分配连续空间
        self.length = 0

        if elements:
            for i, val in enumerate(elements):
                self.elements[i] = val
            self.length = len(elements)

    def get(self, index):
        """ 随机访问：直接通过下标计算地址，效率极高 O(1)"""
        if 0 <= index < self.length:
            return self.elements[index]
```

```
        raise IndexError(" 索引越界！ ")

    def insert(self, index, value):
        """ 插入操作：为了给新元素腾位置，后面的元素都要往后挪 O(n)"""
        if self.length >= self.max_size:
            return " 表已满，无法插入"

        # 从后往前挪动元素
        for j in range(self.length - 1, index - 1, -1):
            self.elements[j + 1] = self.elements[j]

        self.elements[index] = value
        self.length += 1
```

1.2.4 顺序表的优缺点

优点：查找元素非常快（随机存取），不需要额外的存储空间（只存数据）。

缺点：插入和删除操作需要移动大量元素，平均要移动一半的元素；而且空间大小在定义时就要确定，容易造成空间浪费或溢出。

1.3 链表：寻宝游戏

1.3.1 原理分析

链表不要求物理内存连续。每个元素（节点）除了存储自己的数据外，还存了一个“指南针”——指向下一个节点的指针。这样，我们只要知道第一个节点的位置，就可以顺着指针找到所有元素。

这就像一场寻宝游戏：第一个树洞里的纸条告诉你去喷泉找下一条线索，喷泉的线索指向长椅……你不需要知道所有地点的分布，只要顺着线索走就行。

1.3.2 节点定义

在 Python 中，我们可以用一个类来表示节点：

```
class Node:
    def __init__(self, data, next_node=None):
        self.data = data
```

```
        self.next = next_node  # 指向下一个节点的指针

# 快速创建一个链表：1 -> 2 -> 3
first = Node(1)
second = Node(2)
last = Node(3)
first.next = second
second.next = last

# 访问第二个元素的数据
print(f" 第二个元素的数据是: {first.next.data}")
```

1.3.3 插入操作的艺术

在链表中插入一个新节点，不需要像顺序表那样移动后面的元素，只需要修改两个指针：

（1）让新节点的 `next` 指向它要插入位置的后继。

（2）让它前驱的 `next` 指向新节点。

例如，在节点 `first` 和 `second` 之间插入一个新节点 `new_node`：

```
new_node = Node(1.5)
first.next = new_node
new_node.next = second
```

这个操作的时间复杂度是 O(1)，非常快。

1.3.4 为什么需要头节点？

通常在链表的第一个节点之前会设置一个“头节点”，它不存储实际数据，只是为了方便操作。头节点的作用是：当我们在第一个位置插入或删除节点时，不需要单独处理空表的情况，操作逻辑与中间位置完全一致。

1.3.5 链表的优缺点

- **优点**：插入和删除操作非常快（只需修改指针），不需要预先分配连续空间，可以根据需要动态分配内存。
- **缺点**：查找元素需要从头开始逐个遍历（顺序存取），效率低；每个节点除了存储数据外，还要存储指针，存储密度较低。

1.4 更多形式的链表

1.4.1 双向链表

在双向链表中，每个节点不仅知道它的后继，还知道它的前驱。这样，我们可以方便地从一个节点向前或向后遍历。就像浏览器的“前进”和“后退”按钮，你可以随时回到上一页或进入下一页。

节点定义：

```
class DoublyNode:
    def __init__(self, data, prev_node=None, next_node=None):
        self.data = data
        self.prev = prev_node
        self.next = next_node
```

1.4.2 循环链表

在循环链表中，最后一个节点的指针不再指向空，而是指向头节点，形成一个环。这样，我们可以从任意节点出发遍历整个链表。操作系统的轮询调度（Round Robin）就是使用循环链表来实现的，每个进程轮流获得 CPU 时间。

1.5 顺序表与链表的选择

特性	顺序表	链表
存储密度	高（只存数据）	较低（存数据 + 指针）
随机访问	极快 O(1)	慢 O(n)，需要遍历
插入/删除	慢 O(n)，需要移动元素	快 O(1)，只需修改指针
空间分配	静态分配，容易浪费或溢出	动态分配，按需申请

表 1.1: 顺序表与链表的对比

如果经常需要按索引访问元素，或者元素数量固定且变化少，使用顺序表。如果经常插入和删除元素，或者元素数量变化很大，使用链表。

1.6 实战挑战：约瑟夫环问题

约瑟夫环是一个经典问题：n 个人围成一圈，从第一个人开始报数，数到 m 的人出列，然后从下一个人继续报数，直到所有人都出列。求最后的幸存者编号。

这个问题非常适合用循环链表来解决。我们用循环链表模拟围坐的人群，每次数到 m 就删除对应的节点。

```python
def josephus(n, m):
    # 步骤 1: 构建循环链表
    nodes = [Node(i) for i in range(1, n + 1)]
    for i in range(n - 1):
        nodes[i].next = nodes[i + 1]
    nodes[-1].next = nodes[0]  # 首尾相连，形成环

    # 步骤 2: 开始报数
    current = nodes[-1]        # 从头的前一个开始，方便删除
    while current.next != current:
        # 数 m-1 下，停在待删除者的前驱节点
        for _ in range(m - 1):
            current = current.next

        # 删除节点（让当前节点的 next 指向下下个节点）
        print(f" 出局者编号: {current.next.data}")
        current.next = current.next.next

    return current.data

# 例子：9 个人，数到 5 的人出局
winner = josephus(9, 5)
print(f" 最后的幸存者是: {winner}")
```

运行结果（部分输出）：

```
出局者编号：5
出局者编号：1
出局者编号：7
出局者编号：4
出局者编号：3
出局者编号：6
```

出局者编号：9
出局者编号：2
最后的幸存者是：8

1.7 课后作业

为了帮助巩固线性表的知识，下面给出几道练习题。你可以尝试用顺序表和链表分别实现，并比较它们的性能差异。

1. **合并两个有序列表**：给定两个已排序的线性表（例如 $[1, 3, 5]$ 和 $[2, 4, 6]$），将它们合并成一个新的有序线性表。
2. **反转单链表**：实现一个函数，输入一个单链表的头节点，输出反转后的链表。
3. **判断回文链表**：给定一个单链表，判断它是否回文（例如 1->2->2->1 是回文，1->2->3 不是）。
4. **删除链表中倒数第 n 个节点**：给定一个链表，删除倒数第 n 个节点，并返回头节点。
5. **实现循环队列（用顺序表）**：利用顺序表实现一个循环队列，支持入队、出队、判空、判满操作。
6. **用链表实现多项式的加法**：假设多项式用链表表示（每个节点包含系数和指数），实现两个多项式的加法。
7. **找出链表的中间节点**：给定一个单链表，找出它的中间节点。如果节点数为偶数，返回第二个中间节点。
8. **约瑟夫环的数学解法**：除了模拟，约瑟夫环还有一个数学递推公式：$f(1) = 0$，$f(n) = (f(n-1) + m)\%n$。尝试用这个公式直接计算出幸存者，验证结果是否正确。

2 ⟵

栈（Stack）：后进先出的智慧

在学习了通用的“线性表”之后，我们进入一类特殊的线性表——受限的线性表。所谓受限，并不是功能变弱了，而是通过规则的约束，让它在特定场景下变得极其强大。栈就是这样一种结构。

2.1 什么是栈？——后进先出的世界

栈（Stack）是一个特殊的线性表，它只允许在一端进行插入和删除操作。这一端称为栈顶，另一端称为栈底。不含任何元素的栈称为空栈。

2.1.1 核心原则：后进先出（LIFO）

栈的精髓是 LIFO（Last In First Out），即后进先出。最后放入栈中的元素，最先被取出。

生活中的类比：

- 叠盘子：你洗好一个盘子就叠在最上面；要用盘子时也是从最上面拿。最后放上去的盘子最先被拿走。
- 步枪弹夹：子弹一颗一颗压进去（入栈），射击时最上面那颗最先出膛（出栈）。
- 浏览器后退按钮：你访问的页面按顺序被压入一个栈，点击“后退”时，最后访问的页面最先被退出。

2.1.2 为什么是“受限”的？

与普通线性表相比，栈的操作被严格限制在一端。这种限制带来了两个好处：

- 逻辑清晰：我们只关心栈顶元素，无需考虑中间元素的随机访问。
- 实现简单：只需要维护一个栈顶指针，插入和删除操作都集中在同一端。

2.2 栈的基本操作

栈的操作非常简洁，主要就是两个动作：

- 入栈（Push）：将一个元素放到栈顶，成为新的栈顶。
- 出栈（Pop）：取出当前栈顶元素，栈顶指针下移，原来的次顶元素成为新栈顶。

2.2.1 Python 中的栈实现

Python 内置的 `list` 类型就是一个天然的栈：`append()` 对应入栈，`pop()` 对应出栈。我们可以用它来模拟浏览器的前进后退：

```python
# 初始化一个空栈，记录浏览历史
history_stack = []

# 模拟访问页面（入栈）
history_stack.append("首页")
history_stack.append("搜索页")
history_stack.append("详情页")
print(f"当前浏览路径: {history_stack}")

# 模拟点击“返回”按钮（出栈）
if history_stack:
    last_page = history_stack.pop()
    print(f"离开: {last_page}")
    print(f"回到上一页: {history_stack[-1] if history_stack else '空'}")
```

运行这段代码，你会看到“详情页”被弹出，当前页面回到了“搜索页”。这正是后进先出的体现。

2.3 栈的两种存储结构

根据底层存储方式的不同，栈可以分为顺序栈和链栈。

2.3.1 顺序栈

顺序栈使用一组连续的存储单元（如数组）来存放元素，栈顶通常用一个整型变量 `top` 来指示。

- 优点：访问速度快，实现简单。

- 缺点：需要预先分配空间，当栈满时无法再插入新元素，会发生栈溢出（Stack Overflow）。

  小知识："栈溢出"这个词大家可能经常在编程错误中见到，它就是因为递归调用太深或局部数据太大，导致系统分配的栈空间不够用了。

2.3.2 链栈

链栈使用链表结构，每个节点包含数据和指向下一个节点的指针。栈顶就是链表的头节点。

- 优点：动态分配内存，理论上不会满（除非内存耗尽）。
- 缺点：每个节点需要额外的指针空间，访问速度略慢于顺序栈。

现代高级语言（如 Java、Python）的虚拟机在管理方法调用时，底层大量使用了链式栈的思想，以便灵活应对复杂的调用层次。

2.4 栈的经典应用

栈不仅仅是一种存储数据的方式，它代表了一种"保存现场、稍后回来"的思维模式。下面我们看看栈在现实和算法中的几个典型应用。

2.4.1 撤销操作（Undo）

在文字处理软件（如 Word）中，你的每一次编辑（输入文字、删除、格式化）都会被压入一个"撤销栈"。当你按下 Ctrl+Z 时，系统从栈中弹出最近的操作，并执行反向动作，从而恢复到之前的状态。这正是后进先出思想的体现——最后做的操作最先被撤销。

2.4.2 括号匹配

编译器如何判断代码中的括号是否成对匹配？比如 (( )) 是合法的，而 ( ) 是不合法的。算法很简单：

- 从左到右扫描字符串，遇到左括号（(、{、[）就将其入栈。
- 遇到右括号（)、}、]）时，从栈顶弹出一个左括号，检查是否匹配。
- 如果栈为空却遇到右括号，或者最后栈中还有剩余左括号，说明匹配失败。

这个算法只需要一次遍历，时间复杂度 O(n)，空间复杂度 O(n)（最坏情况全是左括号）。很多编程面试题都喜欢考察这个题目。

2.4.3 表达式求值（逆波兰表达式）

计算器在处理 1 + 2 * 3 时，并不是简单地从左到右计算，因为乘法的优先级高于加法。栈可以帮助我们暂存操作数和运算符，从而正确处理优先级。

一种经典方法是先将中缀表达式转换为后缀表达式（逆波兰表达式），然后用一个栈来求值。例如，1 + 2 * 3 对应的后缀表达式是 1 2 3 * +，求值过程如下：

- 遇到操作数 1、2、3 依次入栈。
- 遇到运算符 *，弹出两个操作数 2 和 3，计算 2 * 3 = 6，将 6 入栈。
- 遇到运算符 +，弹出 6 和 1，计算 1 + 6 = 7，得到结果。

整个过程只需要一个栈，非常简洁。

2.4.4 函数调用与递归

在程序运行时，每当调用一个函数，系统就会在内存的“调用栈”上为该函数分配一块区域（称为栈帧），用来存放局部变量、参数和返回地址。当函数执行完毕，对应的栈帧被弹出，控制权交还给调用者。

递归函数正是利用了栈的这种特性：每次递归调用都会压入一个新的栈帧，返回时再弹出。如果递归深度过大，就会导致栈溢出。

2.5 深度探索：迷宫寻路与回溯

栈在人工智能和路径规划中的一个经典应用是迷宫寻路。假设我们在一个二维迷宫中，从起点出发，要找到一条通往终点的路径。我们可以用栈来记录走过的路径，并在遇到死胡同时回溯。

基本思路：

1. 从起点出发，将当前坐标压入栈。
2. 探测四个方向（上、下、左、右）：
    - 如果某个方向可以走（未走过且不是墙），就将新坐标压栈，并标记为已走。
    - 如果所有方向都不可走，说明当前是死胡同，则出栈（回溯）到上一个位置，尝试其他方向。
3. 重复步骤 2，直到到达终点或栈空（无路可走）。

这种“试探-回溯”的过程，正是深度优先搜索（DFS）的核心思想，而栈则是实现这一思想的天然工具。

> 为什么栈适合回溯？
> 因为回溯需要回到最近访问的点，而栈的后进先出特性恰好能记住最近的选择。每一步的决策都被压入栈，当发现此路不通时，只需弹出栈顶，就能回到上一个分叉点。

2.6 栈与队列的对比

在数据结构中，栈和队列经常被放在一起讨论，因为它们都是受限的线性表，但规则恰好相反：

特性	栈（Stack）	队列（Queue）
原则	后进先出（LIFO）	先进先出（FIFO）
操作端	同一端（栈顶）	两端（队尾插入，队头删除）
适用场景	回溯、撤销、递归	排队、任务调度、广度优先搜索

表 2.1: 栈与队列的对比

简单来说，栈是为了“记住过去”（回溯），而队列是为了“保证公平”（先进先出）。

2.7 自测练习

为了巩固对栈的理解，这里给出几道练习题。你可以尝试用笔算，也可以动手写代码验证。

1. **出栈序列判断**
   若入栈序列为 A, B, C，且允许在任意时刻出栈，下列哪个序列是不可能出现的出栈序列？
   A. C, B, A
   B. B, A, C
   C. C, A, B
   D. A, B, C
2. **括号匹配加强版**
   给定一个只包含 (、)、{、}、[、] 的字符串，判断它是否有效。要求使用栈实现。
3. **逆波兰表达式求值**
   输入一个逆波兰表达式（后缀表达式），例如 ["2","1","+","3","*"]，计算其结果。
4. **模拟浏览器的前进后退**
   设计一个浏览器历史记录类，支持访问新页面、后退、前进操作。用两个栈来实现。
5. **最小栈**
   设计一个栈，除了支持常规的 push、pop 操作外，还要能在常数时间内获取栈中的最小元素。
6. **用栈实现队列**
   使用两个栈来实现一个队列，支持入队和出队操作。

7. **回文链表检测**
   给定一个单链表，判断它是否回文（例如 1->2->2->1）。可以借助栈实现：先遍历一遍将值入栈，再第二次遍历时与栈顶比较。
8. **迷宫问题**
   用栈实现一个简单的迷宫求解程序，输出从起点到终点的路径。

本章小结

栈是一种“简约而不简单”的数据结构。它在计算机底层（函数调用）、算法（深度优先搜索）和日常应用（撤销、括号匹配）中无处不在。掌握了栈，你就掌握了程序如何“记住过去”的方法。

在下一章中，我们将学习另一种受限的线性表——队列，看看它又是如何实现“公平”的。